



Neural Nets & Text



Today's Agenda

Natural Language Processing (NLP) Concepts

- Use Cases
- Quick review on bag of words approaches, etc.

Text Pre-processing Functions / Layers

- This implements basic standardization and punctuation removal. It assumes 1-grams, then one-hot encodes.
- No stemming or stop word removal, by default.

Architectures for Sequence Data

- Recurrent Neural Networks (RNNs)
- Bidirectional LSTM



Quick Review of NLP Concepts

Pre-processing Text

- Lower-casing, stop word removal, stemming, removing punctuation, stripping rare tokens, etc.
- Tokenization (this may be chars, words, sentences, etc.
- Integer encoding / indexing the tokens.
- Finally, I may or may not leverage sequence information.
- *Q: what is a bag of words approach? What are n-grams?*

	Database	SQL	Index	Regression	Likelihood	linear
D1	24	21	9	0	0	3
D2	32	10	5	0	3	0
D3	12	16	5	0	0	0
D4	6	7	2	0	0	0
D5	43	31	20	0	3	0
D6	2	0	0	18	7	6
D7	0	0	1	32	12	0
D8	3	0	0	22	4	4
D9	1	0	0	34	27	25

Weighting Term-Documents: TF-IDF

Not all phrases are of equal importance...

- E.g., David less important than Beckham
- If a term occurs all the time, observing its presence is less informative

Inverse-document frequency (IDF) helps address this.

$$\text{IDF} = \log(N/n_j)$$

- Term 'weighting' is then calculated as Term Frequency (TF) x IDF
- n_j = # of docs containing the term, N = total # of docs
- A term is deemed important if it has a high TF and/or a high IDF.
- As TF goes up, the word is more common generally. As IDF goes up, it means very few documents contain this term.

TextVectorization Layer

Pre-processing Text

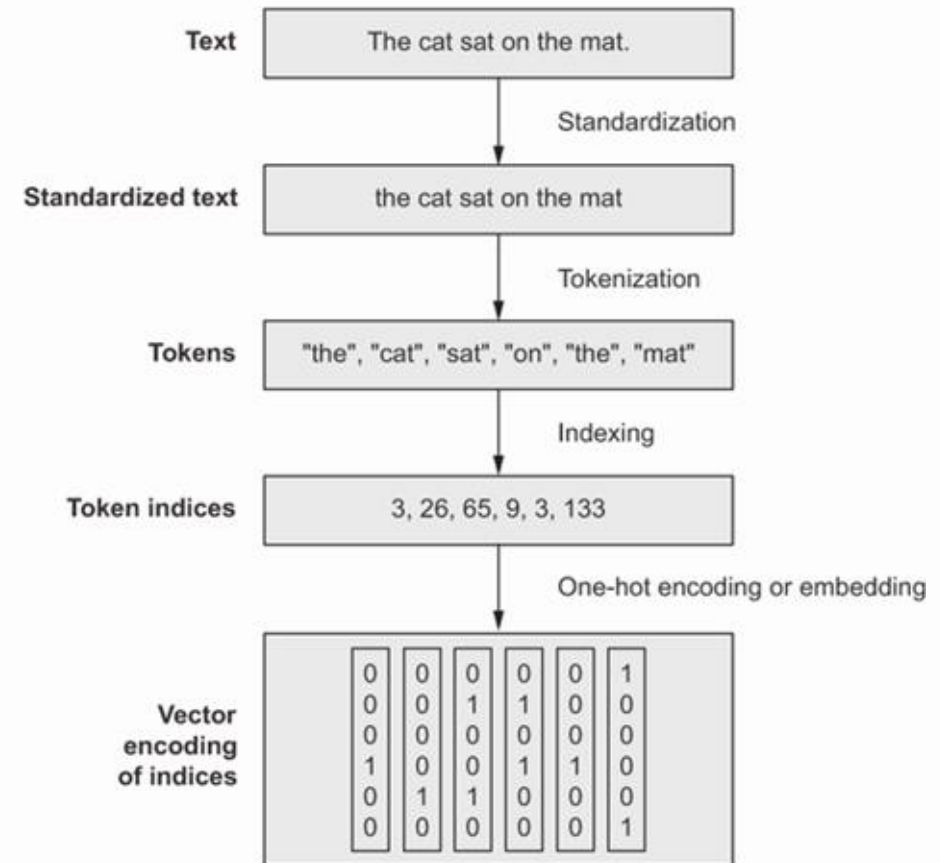
- Standardization, tokenization (words), one-hot-encoding / vectorization.
- The Keras TextVectorization() layer achieves these steps quickly.

Customization

- You can work with n-grams, and do other sorts of pre-processing, using arguments.

Options

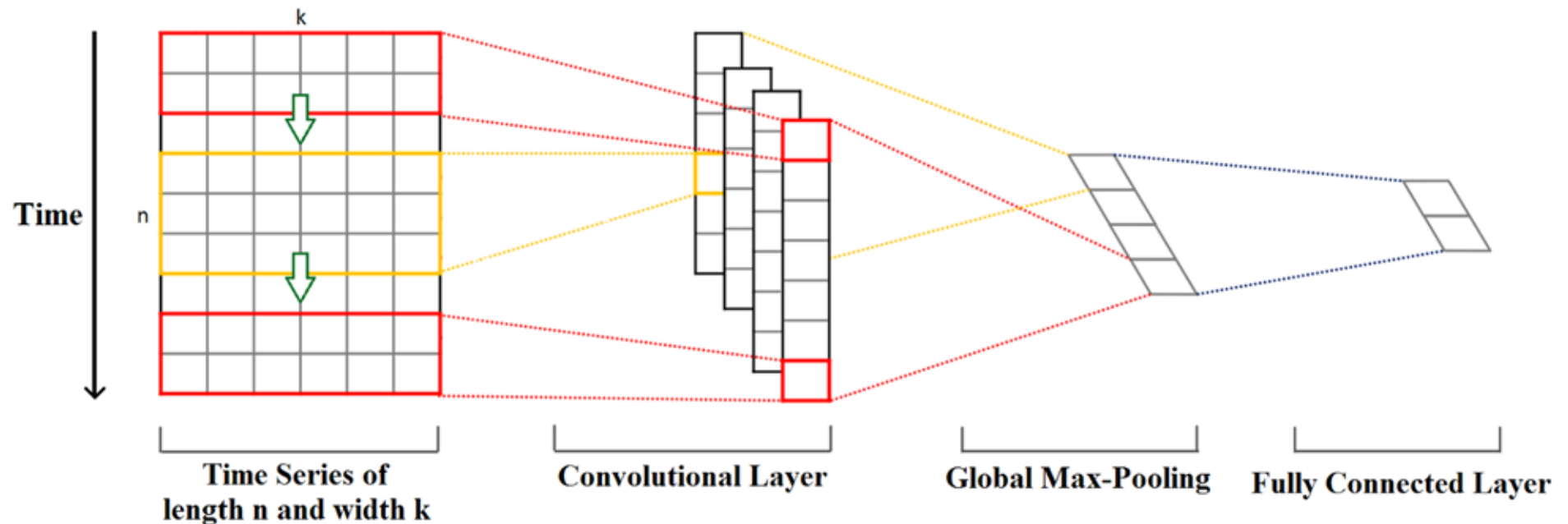
- Include as part of TF Dataset pipeline (more efficient)
- Include as a layer in your Keras model.



Temporal 1D Convolution

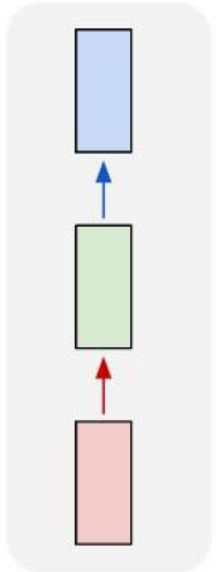
1D Convolution Accomplishes Same Goal as 2D

- It only considers arrangement of features in one dimension (temporal ordering).
- Compresses into shorter sequences, across the entire set of features (just as 2D Conv compresses matrices into smaller matrices, across the entire set of input channels (e.g., RGB)).

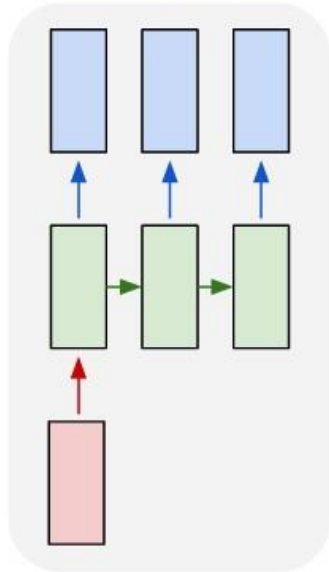


Different Types of NN Task Structures

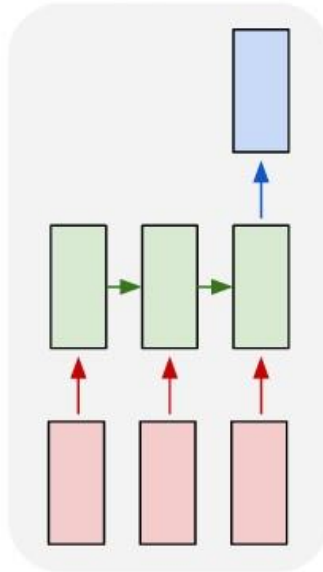
one to one



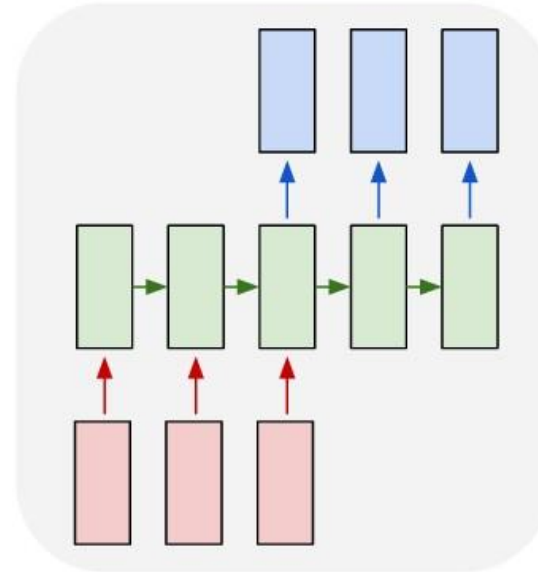
one to many



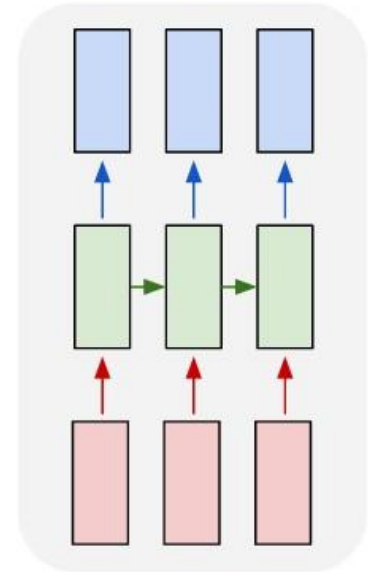
many to one



many to many



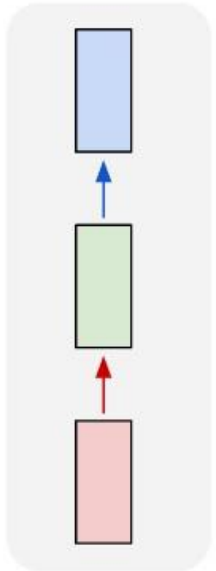
many to many



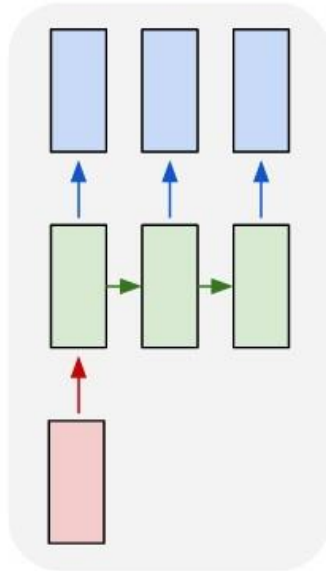
Given what we've learned so far, what challenges can you foresee when it comes to 'many-to-many' problems?

Different Types of NN Task Structures

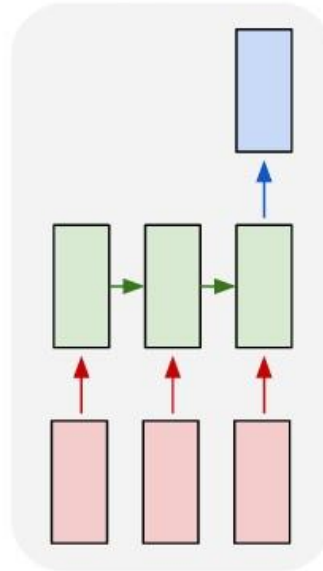
one to one



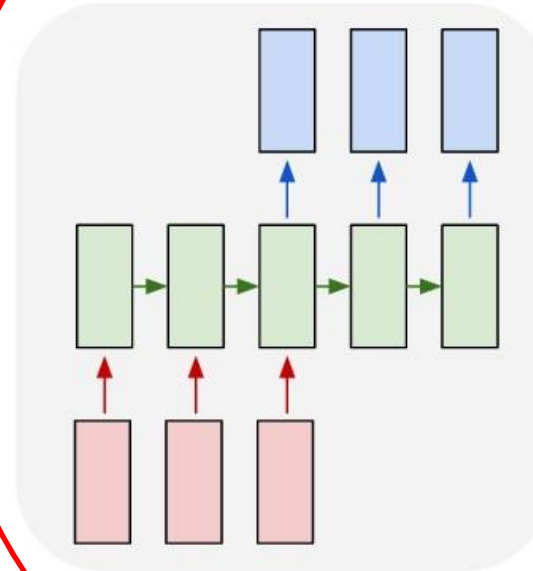
one to many



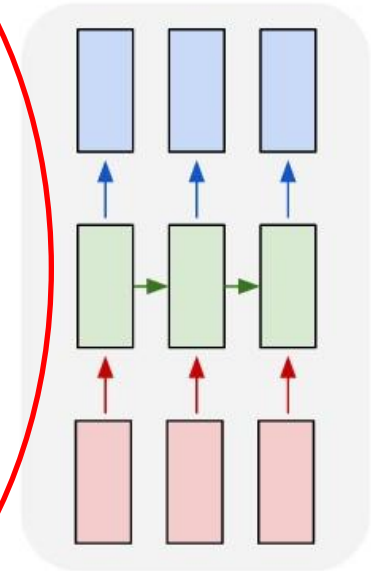
many to one



many to many



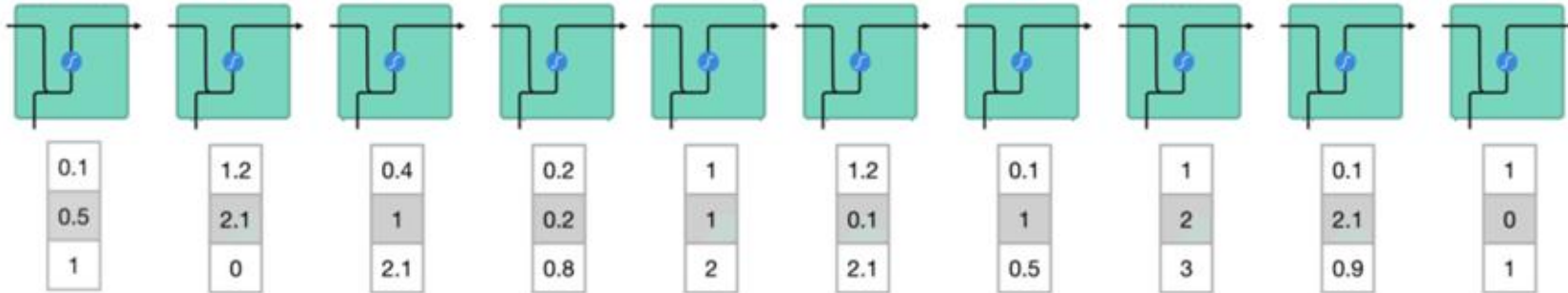
many to many



'Recurrent' Neural Network (RNN)

Keras Layer SimpleRNN

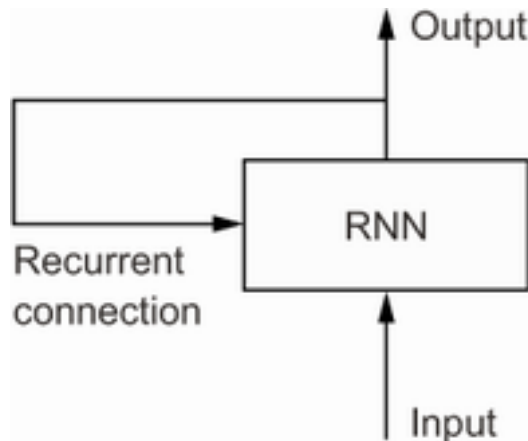
- It's a Dense layer that processes input token by token, with a feedback loop.
- The inputs are arranged to interact with one another based on their temporal sequencing in the data.
- In practice, we cannot use SimpleRNNs to achieve meaningful memory (vanishing gradients arise very quickly! Can you see why?).



How We Often Draw This

Incorporating Memory into a NN

- We include a feedback loop, where output feeds back into the same layer alongside the next input in the sequence (each gets its own separate set of weights, and they are able to interact with one another).



Listing 10.14 More-detailed pseudocode for the RNN

```
1 state_t = 0
2 for input_t in input_sequence:
3     output_t = activation(dot(W, input_t) + dot(U, state_t) + b)
4     state_t = output_t
```

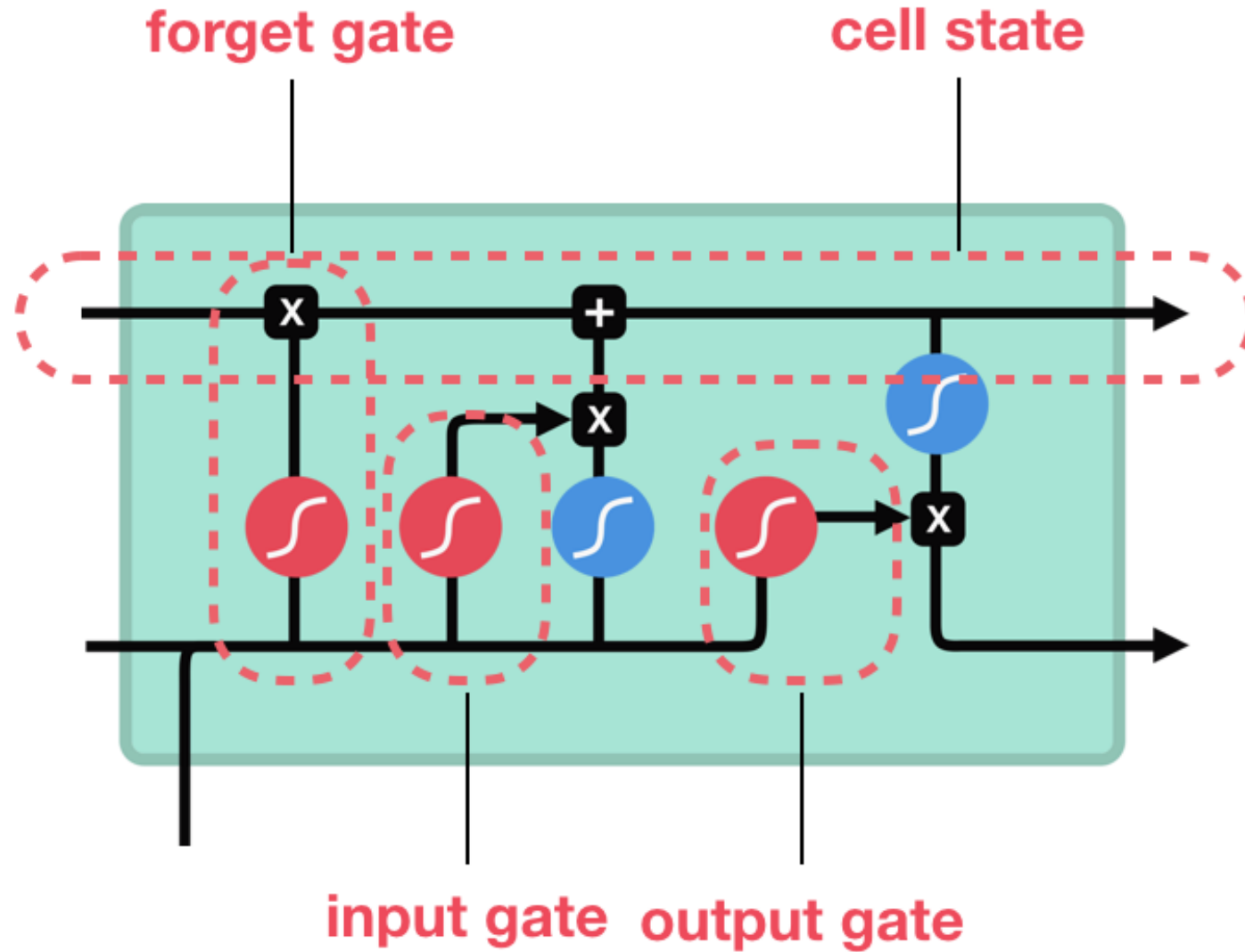
Keras RNN Layers

Long Short-Term Memory (LSTM)

- Addresses vanishing gradient problem with a “carry track” – an additional skip / residual connection that combines output at step t , inputs at step t , and the last carry track’s output.
- The topology means your network can ‘learn’ to use these connections as passthroughs for old info, or it can learn to ‘block’ that information in favor of more recent information (whatever is helpful for accurate prediction).



LSTM



Keras RNN Layers

Long Short-Term Memory (LSTM)

```
>>> inputs = tf.random.normal([32, 10, 8])
>>> lstm = tf.keras.layers.LSTM(4)
>>> output = lstm(inputs)
>>> print(output.shape)
(32, 4)
>>> lstm = tf.keras.layers.LSTM(4, return_sequences=True, return_state=True)
>>> whole_seq_output, final_memory_state, final_carry_state = lstm(inputs)
>>> print(whole_seq_output.shape)
(32, 10, 4)
>>> print(final_memory_state.shape)
(32, 4)
>>> print(final_carry_state.shape)
(32, 4)
```

Keras RNN Layers

Gated Recurrent Unit (GRU)

- Less complex than an LSTM. Combines elements of the LSTM into simpler gated structure.
- Fits more quickly, with less data, but memory tends to be shorter.
- More common in NLP tasks, e.g., because “within sentence” or “within paragraph” memory is often sufficient.



Stacking RNN Layers

Stacked Layers (LSTM or GRUs)

- Same as other 'layer' types, but we need to pass the entire sequence of outputs (not just the last output). We do this with the 'return_sequences=True' argument to the layer.

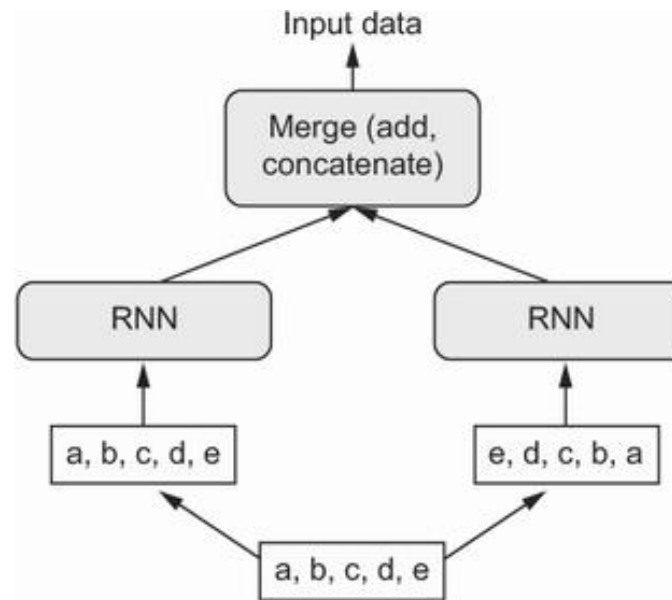
Listing 10.23 Training and evaluating a dropout-regularized, stacked GRU model

```
1 inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
2 x = layers.GRU(32, recurrent_dropout=0.5, return_sequences=True)(inputs)
3 x = layers.GRU(32, recurrent_dropout=0.5)(x)
4 x = layers.Dropout(0.5)(x)
5 outputs = layers.Dense(1)(x)
6 model = keras.Model(inputs, outputs)
```

Bidirectional RNNs

Some Sequences Yield Information in Both Directions

- Consider that, in language, words that come later in a sentence can be predictive of what came before.
- Bidirectional RNNs implement a standard RNN, but they also incorporate a parallel layer implementation that takes the sequence ordered in reverse.



Do We Still Need RNNs?

Some Notable Differences Between RNNs and Transformers

- RNNs can stream output immediately, as the input arrives, whereas a vanilla transformer needs to see the full context/sequence before producing output.
- Transformers are larger neural networks, with more parameters.



Why Do We Need RNNs Though?

Some Notable Differences Between RNNs and Transformers

- RNNs can stream output immediately, as the input arrives, whereas a vanilla transformer needs to see the full context/sequence before producing output.
- Transformers are larger neural networks, with more parameters.

Where RNNs Excel

- Latency! If I need to stream real-time audio or real-time closed captions, etc., then delays are my enemy.
- Edge devices! If I need the ability to use one of these models on-device (e.g., somewhere that lacks internet, or that is privacy sensitive) then it's hard to host a large neural network that has high inference time/costs.

Embedding Layer



With Hot Encodings, Model Will Still Struggle to Figure Out Semantics

- Despite having sequence, the model is “told” that the tokens are orthogonal / independent of one another in their meanings. But that’s not true!

Textual Embedding Layer First Provides Dimensionality Reduction

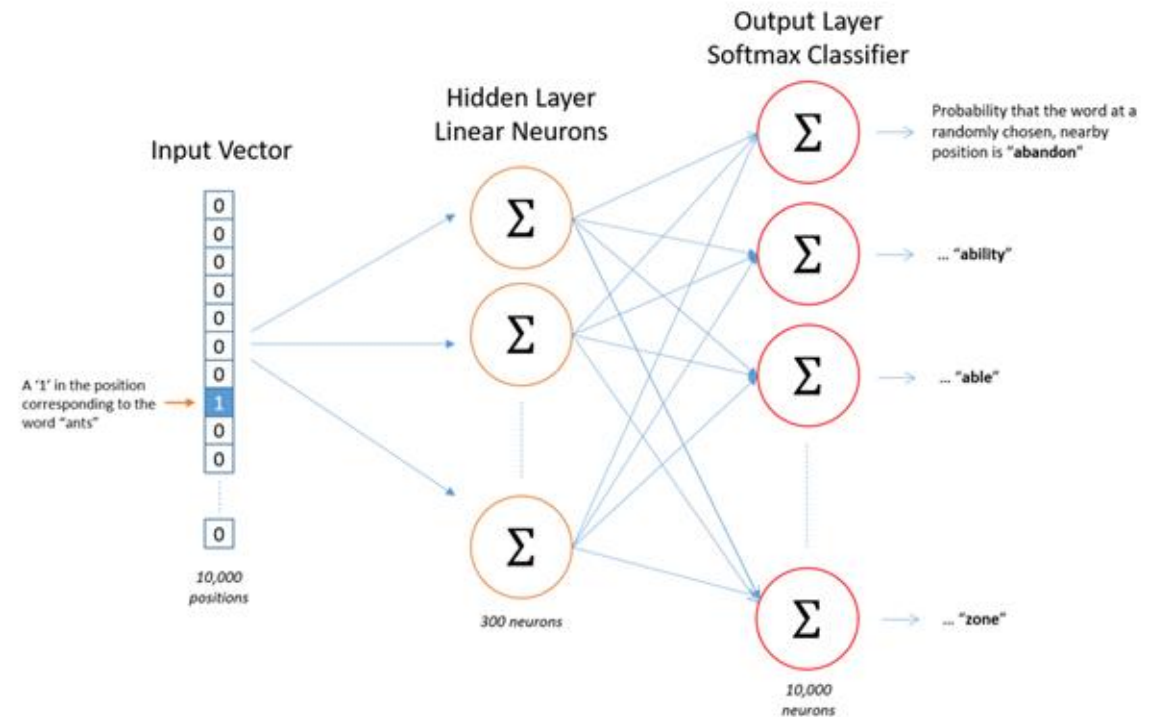
- Represent words into a lower dimensional space – similar vector = similar meaning.
- The Embedding layer is a lookup table that maps tokens to vectors. For each token in the vocabulary, the network learns a vector representation. The vectors are initially random, and the network updates them in training to learn representations that help in prediction (just like with convolution filters!).
- In practice, it is learning semantic relationships...
- This is much better for an RNN than a hot encoding, because 120 values (for example) is $\ll$ 20,000!

Pre-Trained Embeddings: Word2Vec

Word2Vec

- Two types: CBoW and Skipgram
- Construct training examples and labels.

Source Text	Training Samples generated from source text
I will have orange juice and eggs for breakfast	(will, I) (will, have) (will, orange)
I will have orange juice and eggs for breakfast	(have, I) (have, will) (have, orange) (have, juice)
I will have orange juice and eggs for breakfast	(orange, will) (orange, have) (orange, juice) (orange, and)
I will have orange juice and eggs for breakfast	(juice, have) (juice, orange) (juice, and) (juice, eggs)
I will have orange juice and eggs for breakfast	(and, orange) (and, juice) (and, eggs) (and, for)
I will have orange juice and eggs for breakfast	(eggs, juice) (eggs, and) (eggs, for) (eggs, breakfast)
I will have orange juice and eggs for breakfast	(for, and) (for, eggs) (for, breakfast)



Pre-Trained Embeddings: Limitation

Out of Sample Words

- Both GloVe and Word2Vec are limited to words you've seen before in training. They cannot handle new words. Those words thus get omitted / dropped, or you need to do something different.

FastText

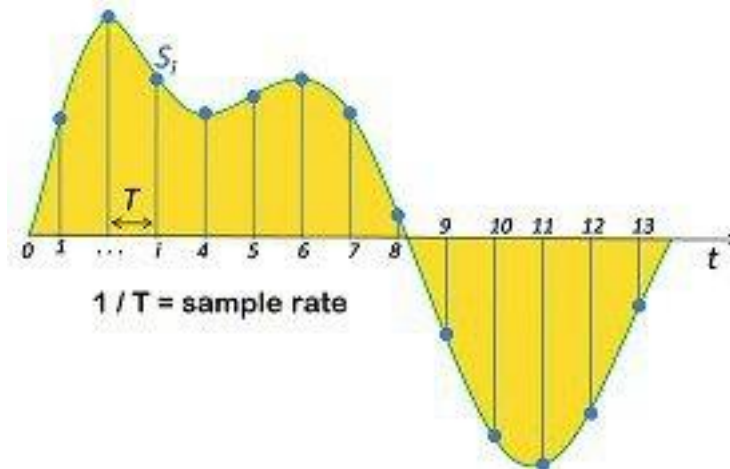
- An extension to Word2Vec which learns character n-grams of words. So, instead of embedding words, we embed portions of words (e.g., a 3-gram character representation would break up the word 'coffee' into 'cof', 'off', 'ffe', ... and then learn vector embeddings of each).



RNN for Audio

Same Sequence Concepts Work for Audio Data

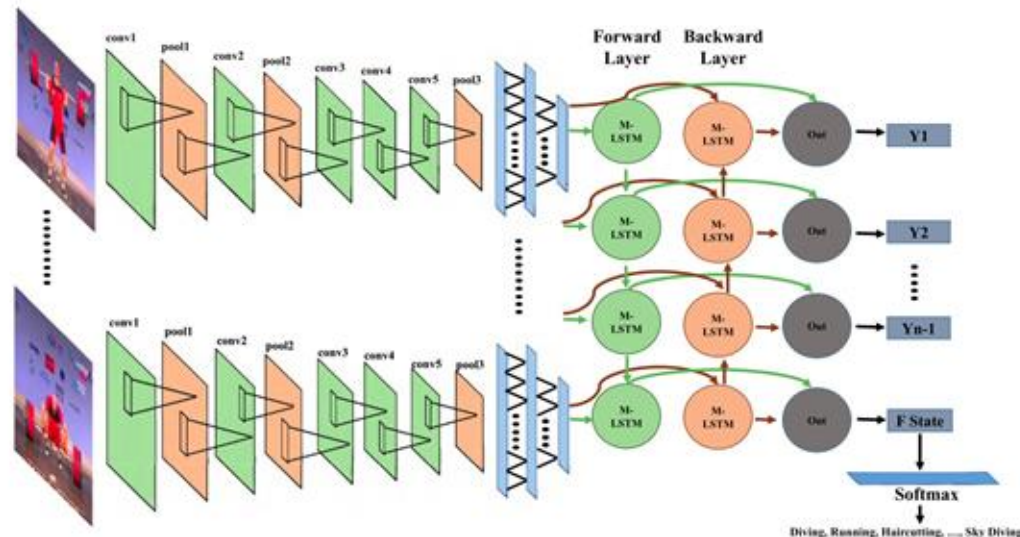
- Audio files are just sequences of numeric values (amplitude), possibly two if it was recorded in stereo.
- Once we recognize this, we realize we can predict things about audio sequences too!



CNN-RNN for Video

Hybrid Topology for Image Sequences

- We Use CNN's to detect features at a given input.
- We feed those feature maps into an RNN architecture, like LSTM.
- We can use this topology to predict things about videos.
- You might pre-process frames using a pre-trained CNN and pass feature maps as sequences to an RNN.



Questions?